

Projet 5

Mission - Déployez votre modèle de Machine Learning

Objectif

Déployer un modèle de machine learning à l'aide d'outils modernes.

- Utilisation de l'intégration continue avec Github, en utilisant de bonnes pratiques concernant les branches de développement et tags de versions
- Analyse de la couverture de nos test
- Utilisation de script de test avant tout merge afin de préserver la qualité du code
- Déploiement continue en utilisant les github actions pour mettre à jour l'API tournant sur Huggingface
- Mise en place d'un suivi des performances de l'API en stockant les prédictions du modèle dans une base de donnée

Modifications apportés au modèle

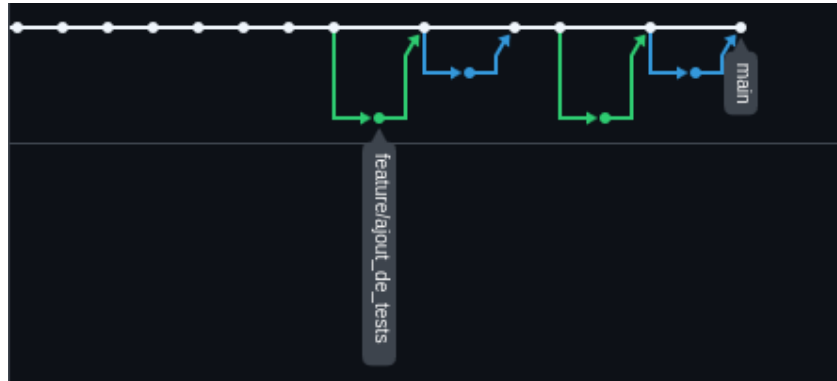
- Encapsulation totale : Intégration du Feature Engineering (SalaryFeatureEngineering, DataCleaningTransformer) directement dans l'objet model.pkl.
- Uniformisation : Suppression du risque d'erreur entre le code d'entraînement et le code de production .
- Allégement du code de l'API qui n'a pas à manipuler en amont les données
- Facilité de mise à jour du modèle, si on modifie la partie feature ingénierie, on met à jour le modèle sans toucher au code de l'API

Performance du modèle

- Sur le set de test le modèle a un f1-score de 0,52
- Une recherche de seuil optimal a été effectuée
- Le seuil optimal a été intégré au modèle (création d'une classe `model_with_threshold` combinant le modèle et son seuil)

Utilisation des branches Github

On crée des branches pour développer de nouvelles fonctionnalités, chaque push déclenche nos tests. Quand ceux-ci sont réussis on peut par exemple merge sur la branche main



Les tests unitaires

- Couverture de 86% sur l'API (app.py). L'entraînement du modèle est volontairement exclu.
- Utilisation de Mocks (Modèle & BDD) pour isoler la logique métier et éviter la pollution des données.

File ▲	statements	missing	excluded	coverage
src/ __init__.py	0	0	0	100%
src/app.py	109	15	0	86%
src/model_With_Threshold.py	10	2	0	80%
src/preprocessing.py	107	37	0	65%
Total	226	54	0	76%

Les tests fonctionnels

- Validation de la "tuyauterie" interne (réception, validation stricte Pydantic, préparation).
- Indépendance de l'environnement : le comportement de l'API est testé en isolation.
- L'unique variable vers la production : l'URL de la base de données (Supabase).

Les tests spécifiques au modèle

- Prévention de la régression mathématique.
- Évaluation continue sur un échantillon fixe (Test Set).
- Alerte immédiate si la fiabilité des prédictions chute lors d'une mise à jour.

Déploiement continu

- Notre branche main est, via les github actions, envoyé vers Huggingface. De là un dockerfile permet de setup l'environnement adéquat pour faire tourner notre API .

L'API

- L'API est généré en utilisant Fast API et pydantic pour contrôler la conformité des inputs.
- Fast API génère automatiquement Swagger, documentant l'API. mkdocs est utilisé pour la documentation globale du projet.

Swagger UI

FastAPI 0.1.0 OAS 3.1

/openapi.json

default

GET / Read Root

POST /predict Predict

Schemas

EmployeeInput ^ Collapse all object

satisfaction_employee_environnement* > Expand all integer [0, 4]

note_evaluation_precedente* > Expand all integer [0, 4]

satisfaction_employee_nature_travail* > Expand all integer [0, 4]

satisfaction_employee_equipe* > Expand all integer [0, 4]

satisfaction_employee_equilibre_pro_perso* > Expand all integer [0, 4]

revenu_mensuel* > Expand all integer > 0

nombre_experiences_precedentes* > Expand all integer ≥ 0

annee_experience_totale* > Expand all integer ≥ 0

annees_dans_l_entreprise* > Expand all integer ≥ 0

Contrôle des prédictions du modèle

- L'API envoie à une base de donnée (PostgreSQL hébergée sur Supabase) les inputs de l'utilisateur de l'API et l'output généré par notre modèle, ainsi que l'heure à laquelle cela s'est passé.
- Ainsi on peut vérifier si les prédictions ont la même répartition que notre data set. Si après quelque mois on peut comparer nos prédictions à la réalité terrain, on peut vérifier que notre modèle à toujours une qualité de prédiction convenable

Contrôle des prédictions du modèle

training_dataset			
Integer	id	PK	Clé Primaire (Auto-incrémentée)
Integer	revenu_mensuel		
Integer	satisfaction_employee_environnement		
String	poste		
String	departement		
String	a_quitte_l_entreprise		Target (Oui/Non)
String	autres_features		...

prediction_logs			
Integer	id	PK	Clé Primaire
DateTime	timestamp		Date et heure de la requête (Par défaut: NOW)
JSON	input_data		Données reçues de l'API (Format flexible)
Integer	prediction_result		Prédiction du modèle (0 ou 1)
Float	prediction_probability		Probabilité associée (ex: 0.85)

*On a une table stockant notre dataset d'entraînement
et un autre permettant de stocker les appels à l'API*